Phương pháp MaxSAT DDD cho bài toán Train Rescheduling: Mô hình và Cài đặt chi tiết

Mục lục

1 Mô hình bài toán Train Rescheduling (TRP) 3
1.1 Mạng lưới đường sắt và đường đi của tàu . . . 3
1.2 Lịch trình và Conflicts . . . 3
1.3 Hàm mục tiêu . . . 3

2 Phương pháp Dynamic Discretization Discovery (DDD) 4
2.1 Ý tưởng chính . . . 4
2.2 Interval Assignment Problem (IAP) . . . 4
2.3 Vòng lặp DDD . . . 4

3 Mô hình MaxSAT cho IAP — Phân loại Hard/Soft 4
3.1 Biến Boolean — Ladder/Unary Delay Encoding . . . 4
3.2 Phân loại Hard Constraints và Soft Constraints . . . 5
3.2.1 Danh sách Hard Clauses . . . 6
3.2.2 Danh sách Soft Clauses . . . 7

4 Encoding hàm mục tiêu 7
4.1 CostTree — Biểu diễn chi phí dưới dạng cây . . . 7

5 Thuật toán RC2 Core-Guided MaxSAT 8
5.1 Nguyên lý RC2 . . . 8
5.2 Xử lý UNSAT Core . . . 8

6 Vòng lặp DDD chính 9
6.1 Local Minimization . . . 10
6.2 Heuristic Thread . . . 10

7 Hai tầng Incremental: SAT Solver vs MaxSAT 10
7.1 Cấu trúc hai vòng lặp . . . 10
7.2 Tầng 1: Incremental SAT Solver . . . 10
7.3 Tầng 2: Incremental MaxSAT (RC2) . . . 11
7.4 Ma trận 4 cấu hình . . . 11
7.4.1 Cấu hình A: Incremental SAT + Incremental MaxSAT (hiện tại) . . . 12
7.4.2 Cấu hình B: Non-Incremental SAT + Incremental MaxSAT . . . 12
7.4.3 Cấu hình C: Incremental SAT + Non-Incremental MaxSAT . . . 12
7.4.4 Cấu hình D: Non-Incremental SAT + Non-Incremental MaxSAT . . . 13
7.5 Tổng hợp so sánh . . . 13
7.6 Đề xuất thực nghiệm . . . 13

8 So sánh hai biến thể encoding **14**

9 Biến đổi sang MaxSAT Binary — Tối ưu bound lớn **14**
9.1 Vấn đề với encoding hiện tại 14
9.2 Ý tưởng: Binary Cost Encoding 14
9.3 Encoding nhị phân chi tiết 15
9.4 Cách tiếp cận khả thi nhất: Binary CostTree 16
9.5 Tác động lên RC2 17

10 Thay thế MaxSAT bằng Incremental SAT + SCPB **18**
10.1 Tại sao không dùng Totalizer? 18
10.2 SCPB Encoding 18
10.3 Điều khiển bound qua Assumptions 19
10.4 Áp dụng cho hàm mục tiêu 19
10.5 Tích hợp với DDD 20
10.6 Hybrid SCPB (SCPB^h) 21
10.7 So sánh RC2 vs SAT + SCPB 21
10.8 Generalized Totalizer Encoding (GTE) 21

11 Tổng kết **23**

1 Mô hình bài toán Train Rescheduling (TRP)

1.1 Mạng lưới đường sắt và đường đi của tàu

Mạng lưới đường sắt gồm tập R các *track segment* (đoạn đường ray). Mỗi tàu $i \in I$ có một *rail path* $R_i \subseteq R$ là dãy các track segment liên tiếp mà tàu đi qua. Với mỗi $r \in R_i$, ký hiệu:

- l_i^r: thời gian chạy tối thiểu (*travel time*) của tàu i trên segment r.
- $\underline{t}^{ir}$: thời điểm sớm nhất (*earliest time*) tàu i có thể vào segment r.

Trong cài đặt (file `problem.rs`), mỗi tàu được biểu diễn bởi struct `Train` gồm danh sách các `Visit`:

```
pub struct Train { pub visits: Vec<Visit> }
pub struct Visit {
    pub resource_id: usize,     // ID cua track segment (resource)
    pub earliest: i32,          // t_ir (earliest entry time)
    pub aimed: Option<i32>,     // thoi diem mong muon (tinh cost)
    pub travel_time: i32,       // l^r (thoi gian chay toi thieu)
}
```

1.2 Lịch trình và Conflicts

Định nghĩa 1 (Lịch trình khả thi)**.** *Một* lịch trình $t = \{t^{ir} \mid i \in I, r \in R_i\}$ *gán thời điểm vào cho mỗi visit. Lịch trình khả thi phải thỏa mãn:*

(i) ***Lower bound:*** $t^{ir} \geq \underline{t}^{ir}$ *với mọi* i, r.

(ii) ***Travel time (precedence):*** $t^{iq} \geq t^{ir} + l^r$ *với mọi* $i \in I$, $r \prec_i q$.

(iii) ***Disjunctive (resource conflict):*** *Với mọi cặp* (i, r) *và* (j, q) *sử dụng chung resource, phải có:* $t^{jq} \geq t^{ir} + l^{rq}$ *hoặc* $t^{ir} \geq t^{jq} + l^{qr}$.

Trong cài đặt, các cặp resource có thể conflict được lưu trong `Problem.conflicts: Vec<(usize, usize)>`. Đa số các instance chỉ có conflict giữa resource với chính nó (hai tàu không chiếm cùng segment cùng lúc).

1.3 Hàm mục tiêu

Mục tiêu là tối thiểu hóa tổng delay cost $\sum_{i \in I} c^{if}$. Cài đặt hỗ trợ nhiều loại cost qua enum `DelayCostType`:

1. **Stepwise hữu hạn** (`FiniteSteps`): hàm bậc thang với ngưỡng cố định, ví dụ:

$$c^{if} = \begin{cases} 3 & d > 360s \\ 2 & d > 180s \\ 1 & d > 0 \\ 0 & d = 0 \end{cases}$$

2. **Stepwise vô hạn** (`InfiniteSteps`): $c^{if} = \lceil d/Q \rceil$ với bước Q (60s, 180s, hoặc 360s).
3. **Liên tục** (`Continuous`): $c^{if} = \max(0, t^{if} - \underline{t}^{if})$.

Trong đó $d = \max(0,\ t^{if} - \text{aimed}^{if})$ là delay so với thời gian mong muốn.

2 Phương pháp Dynamic Discretization Discovery (DDD)

2.1 Ý tưởng chính

Thay vì sử dụng full Time-Indexed formulation (quá lớn) hay Big-M formulation (LP relaxation yếu), DDD giải một chuỗi các bài toán TI *nhỏ hơn* bằng cách chia planning horizon thành các interval có kích thước khác nhau.

2.2 Interval Assignment Problem (IAP)

Với mỗi $i \in I, r \in R_i$, chia feasibility interval $[\underline{t}^{ir}, M)$ thành partition $\Lambda^{ir} = \{\lambda_1^{ir}, \lambda_2^{ir}, \ldots\}$ với $\lambda_p^{ir} = [h_p^{ir}, h_{p+1}^{ir})$.

Định nghĩa 2 (Gán interval khả thi)**.** *Hàm $S : \Omega \to \Lambda$ gán cho mỗi (i, r) một interval $S(i, r) \in \Lambda^{ir}$ là* khả thi *nếu không có cặp interval nào incompatible.*

Định nghĩa 3 (Incompatibility)**.** *Hai interval λ_p^{ir} và $\lambda_{p'}^{jq}$ là:*

- ***Rail path incompatible*** *(cùng tàu $i = j$, $r \prec_i q$) nếu: $h_p^{ir} + l^r > h_{p'+1}^{iq}$.*
- ***Conflict incompatible*** *(khác tàu $i \neq j$) nếu: $h_{p'}^{jq} + l^{qr} > h_{p+1}^{ir}$* ***và*** *$h_p^{ir} + l^{rq} > h_{p'+1}^{jq}$.*

Định lý 1. *Nếu TRP có lời giải tối ưu t^*, thì IAP có lời giải tối ưu S^* với $\bar{c}(S^*) \leq c(t^*)$. Tức giá trị tối ưu của IAP là* ***lower bound*** *cho TRP.*

2.3 Vòng lặp DDD

1. **Khởi tạo:** Mỗi (i, r) chỉ có 1 interval $[\underline{t}^{ir}, M)$.
2. **Giải IAP:** Tìm interval assignment S_k^* tối ưu. Tính schedule $\tau_k^* = \Phi(S_k^*)$ bằng cách gán mỗi tàu vào thời điểm đầu interval.
3. **Kiểm tra:** Nếu τ_k^* khả thi $\to$ tối ưu. Nếu có vi phạm, *refine* bằng cách split interval tại điểm conflict.

3 Mô hình MaxSAT cho IAP — Phân loại Hard/Soft

IAP chỉ chứa biến nhị phân, nên có thể chuyển sang SAT/MaxSAT. Phần này mô tả chi tiết mô hình, **phân biệt rõ hard constraints và soft constraints**.

3.1 Biến Boolean — Ladder/Unary Delay Encoding

Đây là biến thể **chính** được sử dụng (file `maxsatddd_ladder.rs`). Mỗi visit (i, r) có một *chuỗi delay* gồm các biến Boolean sắp xếp theo thời gian tăng dần:

$$d_0^{ir} \Rightarrow d_1^{ir} \Rightarrow d_2^{ir} \Rightarrow \cdots \Rightarrow d_n^{ir}$$

trong đó $d_k^{ir} = 1$ nghĩa là "tàu i vào resource r tại thời điểm $\geq t_k$" (tức là bị delay ít nhất đến t_k). Đây là chuỗi *monotone giảm*: nếu delay đến t_k thì chắc chắn delay đến mọi t_j với $j < k$.

Cấu trúc dữ liệu Occ. Mỗi visit được lưu trong struct `Occ`:

```
struct Occ<L: satcoder::Lit> {
    cost: Vec<Bool<L>>,          // cost variables
    cost_tree: CostTree<L>,      // cay chi phi
    delays: Vec<(Bool<L>, i32)>, // day cac (bien_delay, thoi_diem)
    incumbent_idx: usize,        // vi tri loi giai hien tai
}
```

Trường `delays` chứa danh sách $[(d_0, t_0), (d_1, t_1), \ldots, (d_n, \infty)]$ với:

- $d_0 =$ `true` (luôn delay ít nhất đến earliest time).
- $d_n =$ `false` (sentinel, không bao giờ delay đến ∞).
- Clause ràng buộc thứ tự: $d_{k+1} \Rightarrow d_k$ (tức $\overline{d_{k+1}} \vee d_k$).

Ngữ nghĩa. "Tàu i vào resource r trong interval $[t_k, t_{k+1})$" tương đương với $d_k = 1 \wedge d_{k+1} = 0$. Vị trí `incumbent_idx` chỉ ra interval hiện tại: $d_{\text{idx}} = 1$ và $d_{\text{idx}+1} = 0$.

Thêm time point mới. Khi DDD cần refine (chia interval), hàm `time_point()` chèn biến mới vào giữa chuỗi delay:

Algorithm 1 Thêm time point t vào chuỗi delay của visit

```
function TimePoint(solver, t)
    idx ← vị trí sao cho t_{idx−1} < t ≤ t_idx
    if t_idx == t then return (d_idx, false)          ▷ đã tồn tại
    end if
    v ← NewVar(solver)
    Chèn (v, t) vào vị trí idx
    Thêm hard clause: v̄ ∨ d_{idx−1}                   ▷ v ⇒ d_{idx−1}
    Thêm hard clause: d̄_{idx+1} ∨ v                   ▷ d_{idx+1} ⇒ v
    return (v, true)
end function
```

Mỗi biến mới chỉ cần **2 hard clause** để duy trì tính monotone ($d_{k+1} \Rightarrow d_k$), thay vì $O(n)$ clause cho exactly-one trong one-hot encoding.

3.2 Phân loại Hard Constraints và Soft Constraints

Mô hình MaxSAT gồm hai loại clause:

Hard Clause

Hard clauses — *phải* được thỏa mãn trong mọi lời giải. Vi phạm hard clause nghĩa là lịch trình không khả thi.

Soft Clause

Soft clauses — *mong muốn* được thỏa mãn, mỗi clause có weight $w > 0$. MaxSAT tối thiểu hóa tổng weight các soft clause bị vi phạm.

3.2.1 Danh sách Hard Clauses

1. **H1 — Delay chain (thứ tự thời gian):**

   Hard Clause

   $\overline{d_{k+1}^{ir}} \vee d_k^{ir}$ cho mỗi cặp thời điểm liên tiếp $t_k < t_{k+1}$ của visit (i, r).

   Ý nghĩa: nếu tàu delay ít nhất đến t_{k+1} thì chắc chắn delay ít nhất đến t_k.

2. **H2 — Travel time (precedence):**

   Hard Clause

   $\overline{d_t^{i,v}} \vee d_{t+l^v}^{i,v+1}$ khi phát hiện vi phạm travel time.

   Ý nghĩa: nếu tàu i delay đến t tại visit v, thì tại visit $v+1$ phải delay ít nhất đến $t + l^v$.

3. **H3 — Resource conflict (disjunctive):**

   Hard Clause

   $\overline{d_{t_{1,\text{out}}}^{v_1}} \vee \overline{d_{t_{2,\text{out}}}^{v_2}} \vee d_{t_{2,\text{out}}}^{v_1} \vee d_{t_{1,\text{out}}}^{v_2}$

   Ý nghĩa: khi hai tàu chiếm cùng resource và overlap, ít nhất một tàu phải delay để tránh nhau.

4. **H4 — Cost implication:**

   Hard Clause

   $\overline{d_t^{ir}} \vee y_c^{ir}$ khi $\text{delay_cost}(i, r, t) = c > 0$.

   Ý nghĩa: nếu tàu i delay đến t tại visit r, thì chi phí ít nhất c.

5. **H5 — Cost ordering:**

   Hard Clause

   $\overline{y_{c'}^{ir}} \vee y_c^{ir}$ với $c' > c$.

   Ý nghĩa: chi phí $\geq c'$ implies chi phí $\geq c$.

6. **H6 — Totalizer clauses (RC2):**

   Hard Clause

   Các clause nội bộ của Totalizer tree, encode ràng buộc $\sum \overline{l_i} \leq k$.

   Ý nghĩa: xuất hiện khi RC2 xử lý UNSAT core, giới hạn số soft clause cùng bị vi phạm.

3.2.2 Danh sách Soft Clauses

1. **S1 — Cost minimization:**

   Soft Clause

   $(\overline{y_c^{ir}},\ w = \Delta c)$ cho mỗi mức cost c của mỗi visit (i, r).

   Ý nghĩa: "không muốn chi phí $\geq c$", weight $= \Delta c$ là chênh lệch chi phí giữa mức c và mức thấp hơn gần nhất.

2. **S2 — Totalizer bound (RC2):**

   Soft Clause

   $(\overline{\text{tot.rhs}[k]},\ w = w_{\min})$ từ RC2 core processing.

   Ý nghĩa: "tổng số lần vi phạm trong core $\leq k$", weight là min weight trong core.

Tóm tắt. Bảng phân loại:

ID	Loại	Hard/Soft	Nguồn gốc
H1	Delay chain	**Hard**	Khởi tạo & refine
H2	Travel time	**Hard**	Travel time conflict
H3	Resource conflict	**Hard**	Resource conflict
H4	Cost implication	**Hard**	CostTree.add_cost
H5	Cost ordering	**Hard**	CostTree.add_cost
H6	Totalizer internal	**Hard**	RC2 core processing
S1	Cost minimization	**Soft**	CostTree.add_cost
S2	Totalizer bound	**Soft**	RC2 core processing

Bảng 1: Phân loại hard/soft constraints trong mô hình MaxSAT

Nhận xét quan trọng.

- Tất cả ràng buộc *vật lý* (travel time, resource conflict, thứ tự thời gian) là **hard** — không thể vi phạm.
- Chỉ các ràng buộc *chi phí* (delay cost) là **soft** — solver cố gắng giảm thiểu tổng cost nhưng có thể "chấp nhận" delay.
- RC2 tạo thêm cả hard (Totalizer internal) lẫn soft (Totalizer bound) trong quá trình giải.

4 Encoding hàm mục tiêu

4.1 CostTree — Biểu diễn chi phí dưới dạng cây

Hàm mục tiêu được encode bằng cấu trúc `CostTree` (file `costtree.rs`). Mỗi visit có một cost tree riêng. Ý tưởng:

- Mỗi node trong cây ứng với một *mức chi phí* c và một biến Boolean y_c.

- $y_c = 1$ nghĩa là "chi phí tại visit này $\geq c$".
- Các node tạo thành chuỗi implication: $y_{c'} \Rightarrow y_c$ với $c < c'$ (chi phí cao hơn $\Rightarrow$ chi phí thấp hơn chắc chắn đạt) — đây là **hard clause H5**.
- Mỗi biến y_c tạo **soft clause S1**: $\overline{y_c}$ với weight $= \Delta c$.

Ví dụ. Với stepwise cost $\{0, 1, 2, 3\}$ tại ngưỡng $\{0, 180s, 360s\}$:

- y_1: chi phí ≥ 1. Soft: $\overline{y_1}$, weight 1.
- y_2: chi phí ≥ 2. Hard: $y_2 \Rightarrow y_1$. Soft: $\overline{y_2}$, weight 1.
- y_3: chi phí ≥ 3. Hard: $y_3 \Rightarrow y_2$. Soft: $\overline{y_3}$, weight 1.

Khi một time point mới t có delay cost c được thêm, cost tree tạo:

- Hard clause H4: $\overline{d_t} \vee y_c$ (nếu tàu delay đến t, thì chi phí ít nhất c).
- Soft clause S1: $\overline{y_c}$ với weight $= \Delta c$ (nếu mức c là mới).

5 Thuật toán RC2 Core-Guided MaxSAT

Cài đặt sử dụng thuật toán *RC2 (Relaxable Cardinality Constraints)* để giải MaxSAT.

5.1 Nguyên lý RC2

Algorithm 2 RC2 MaxSAT Loop (tích hợp trong DDD)

```
total_cost ← 0
soft_constraints ← HashMap từ literal → (loại, weight, weight_gốc)
loop
    Sắp xếp assumptions theo weight giảm dần
    result ← SAT solver với assumptions (lấy tối đa n_assumps literal)
    if SAT và n_assumps < |soft_constraints| then
        n_assumps ← n_assumps + 20                    ▷ tăng dần số assumptions
        continue
    else if SAT then
        Cập nhật incumbent_idx cho mỗi visit
        Thực hiện local minimization
        break (chuyển sang conflict detection)
    else if UNSAT (có core) then
        Xử lý core (xem mục 5.2)
    end if
end loop
```

5.2 Xử lý UNSAT Core

Khi solver trả về UNSAT core $C = \{l_1, l_2, \ldots, l_m\}$:

1. Tính $w_{\min} = \min_{l \in C} \text{weight}(l)$.
2. total_cost $\leftarrow$ total_cost $+ w_{\min}$.
3. Với mỗi $l \in C$:

- Giảm weight: $\text{weight}(l) \leftarrow \text{weight}(l) - w_{\min}$.
- Nếu weight $= 0$:
  - `Delay`: loại bỏ khỏi soft constraints.
  - `Totalizer(tot, bound)`: tăng bound, tạo soft S2 mới.

4. Nếu $|C| > 1$: tạo **Totalizer** mới (tạo thêm hard H6):

$$\text{tot} = \text{Totalizer::count}(\overline{l_1}, \overline{l_2}, \ldots, \overline{l_m}, \text{bound} = 1)$$

   Thêm soft S2: $\overline{\text{tot.rhs}[1]}$ ("tổng ≤ 1") với weight $w_{\min}$.

5. Nếu $|C| = 1$: thêm hard clause $\overline{l_1}$.

Totalizer. Là cấu trúc cây (sorting network) encode ràng buộc cardinality: $\sum \overline{l_i} \leq k$. Bên trong Totalizer có nhiều hard clauses H6. Khi cần tăng bound $k \to k+1$, totalizer thêm hard clauses mới qua `increase_bound()`.

Incremental assumptions. Cài đặt sử dụng kỹ thuật *partial assumptions*: ban đầu chỉ gửi 20 soft literals có weight cao nhất cho solver. Nếu SAT, tăng thêm 20 literals cho đến khi tất cả được bao phủ. Điều này giúp solver tìm core nhỏ hơn ở giai đoạn đầu.

6 Vòng lặp DDD chính

Algorithm 3 DDD-TRP Main Loop (Ladder encoding variant)

```
Khởi tạo: mỗi visit có 1 delay variable d_0 = true tại t = earliest
loop
    if timeout then return lỗi timeout
    end if
    if last iteration was SAT then                    ▷ — Conflict Detection Phase —
        for all visit có thay đổi do
            Travel time check:
            if t_in^v + l^v > t_in^{v+1} then
                Tạo time point mới t_new = t_in^v + l^v tại visit v + 1
                Thêm hard H2: d_{t_in}^v ⇒ d_{t_new}^{v+1}
            end if
            Resource conflict check:
            for all tàu khác trên cùng resource do
                if occupation intervals overlap then
                    Tạo time points mới, thêm hard H3: disjunctive 4-literal
                end if
            end for
        end for
        if không có conflict then
            Extract solution, verify, return optimal
        end if
    end if
    Xử lý time points mới: thêm hard H4, H5 và soft S1
    RC2 solve: call SAT solver, nếu UNSAT: thêm hard H6 và soft S2
    Nếu SAT: cập nhật incumbent, local minimize
end loop
```

6.1 Local Minimization

Sau khi tìm được lời giải SAT, cài đặt thực hiện *local minimization*: lặp qua tất cả visits và thử giảm `incumbent_idx` (chọn interval sớm hơn) nếu không gây conflict mới. Quá trình này giảm delay không cần thiết mà MaxSAT solver có thể tạo ra do laziness trong conflict generation.

6.2 Heuristic Thread

Cài đặt chạy một *heuristic thread* song song (sử dụng BigM solver với Gurobi) để tìm upper bound. Khi lower bound (từ MaxSAT) bằng upper bound (từ heuristic), giải thuật kết thúc sớm mà không cần chứng minh tính tối ưu hoàn toàn qua MaxSAT.

7 Hai tầng Incremental: SAT Solver vs MaxSAT

Thuật toán hiện tại có **hai vòng lặp lồng nhau**, mỗi vòng tương ứng với một tầng "incremental" khác nhau. Hai tầng này hoàn toàn độc lập và có thể được cấu hình riêng biệt.

7.1 Cấu trúc hai vòng lặp

Algorithm 4 Cấu trúc lồng nhau: DDD loop ⊃ RC2 loop ⊃ SAT call

```
solver ← NewSATSolver()                                   ▷ SAT solver instance
loop                                                      ▷ Vòng ngoài: DDD iteration
    Phát hiện conflicts → thêm hard clauses H1–H3 vào solver
    Thêm time points → thêm hard H4, H5 và soft S1 vào solver
    loop                                                  ▷ Vòng trong: RC2 MaxSAT loop
        result ← solver.solve_with_assumptions(soft literals)   ▷ SAT call
        if SAT then
            Cập nhật incumbent → break sang DDD iteration mới
        else if UNSAT (core) then
            Thêm hard H6 (Totalizer) và soft S2 vào solver
            total_cost += w_min                           ▷ tiếp tục RC2 loop
        end if
    end loop
end loop
```

7.2 Tầng 1: Incremental SAT Solver

Câu hỏi: Có nên *tái sử dụng* cùng SAT solver instance giữa các lần gọi `solve_with_assumptions()`?

- **Incremental SAT** (hiện tại): Một SAT solver duy nhất được tạo lần đầu và sử dụng cho *tất cả* SAT calls — cả trong RC2 loop lẫn qua các DDD iterations. Clauses chỉ được *thêm* vào, không bao giờ bị xóa.
- **Non-Incremental SAT**: Tạo SAT solver mới trước mỗi (hoặc một số) SAT call. Phải add lại *toàn bộ* hard clauses hiện có.

SAT solver hỗ trợ incremental. Trong cài đặt, SAT solver cung cấp interface `SatSolverWithCore` với hai phương thức chính:

- `add_clause(clause)`: thêm hard clause, **tồn tại vĩnh viễn**.

Tiêu chí	Incremental SAT	Non-Incremental SAT
Learned clauses	Giữ qua các SAT calls	Mất mỗi lần tạo mới
Add clauses	Chỉ thêm clauses mới	Phải add lại tất cả
Preprocessing	Chỉ lần đầu	Có thể mỗi lần
Clause database	Tích lũy, ngày càng lớn	Sạch sẽ mỗi lần
Warm start	Có	Không

Bảng 2: Incremental vs Non-Incremental ở tầng **SAT solver**

- `solve_with_assumptions(assumptions)`: giải SAT dưới tập assumptions, trả về SAT (model) hoặc UNSAT (core). Assumptions chỉ có hiệu lực cho lần gọi hiện tại.

Các SAT solver hiện đại như CaDiCaL, Lingeling đều hỗ trợ incremental qua assumptions, giữ learned clauses giữa các lần gọi `solve`.

7.3 Tầng 2: Incremental MaxSAT (RC2)

Câu hỏi: Có nên *giữ lại trạng thái RC2* (total_cost, soft_constraints, Totalizers) giữa các DDD iteration?

- **Incremental MaxSAT** (hiện tại): Trạng thái RC2 (bao gồm total_cost, HashMap soft_constraints, các Totalizer đã tạo) được giữ nguyên qua các DDD iteration. Khi DDD thêm clauses mới (H2, H3 từ conflicts), RC2 *tiếp tục* từ trạng thái hiện tại.
- **Non-Incremental MaxSAT**: Reset toàn bộ trạng thái RC2 tại mỗi DDD iteration: tạo lại soft_constraints từ đầu, xóa Totalizers cũ, đặt total_cost = 0. RC2 giải lại bài toán MaxSAT từ đầu với tập clauses mới.

Tiêu chí	Incremental MaxSAT	Non-Incremental MaxSAT
RC2 state	Giữ qua DDD iters	Reset mỗi DDD iter
Totalizers	Tích lũy	Tạo mới
total_cost	Tăng dần	Bắt đầu từ 0
Số SAT calls/iter	Ít (warm start)	Nhiều (giải lại từ đầu)
LB quality	Monotone tăng	Có thể dao động

Bảng 3: Incremental vs Non-Incremental ở tầng **MaxSAT (RC2)**

7.4 Ma trận 4 cấu hình

Kết hợp hai tầng tạo ra **4 cấu hình** có thể thử nghiệm:

	Incremental SAT	**Non-Incremental SAT**
Incremental MaxSAT	A: Hiện tại	B
Non-Incremental MaxSAT	C	D

Bảng 4: 4 cấu hình kết hợp hai tầng incremental

7.4.1 Cấu hình A: Incremental SAT + Incremental MaxSAT (hiện tại)

- Một SAT solver duy nhất, tồn tại xuyên suốt.
- RC2 state giữ nguyên qua các DDD iteration.
- Ưu: nhanh nhất về overhead, giữ learned clauses + Totalizers.
- Nhược: clause database rất lớn, Totalizers cũ có thể không còn liên quan.

7.4.2 Cấu hình B: Non-Incremental SAT + Incremental MaxSAT

- Tạo SAT solver mới cho mỗi SAT call (hoặc mỗi DDD iteration).
- RC2 state (soft_constraints, Totalizers, total_cost) vẫn giữ, nhưng phải add lại tất cả hard clauses + Totalizer clauses vào solver mới.
- Ưu: preprocessing fresh, không tích lũy learned clauses stale.
- Nhược: chi phí rebuild cao, mất warm start.

Algorithm 5 Cấu hình B: Non-Incremental SAT + Incremental MaxSAT

```
clause_store ← []                                   ▷ lưu tất cả clauses bên ngoài solver
rc2_state ← InitRC2                                 ▷ giữ xuyên suốt
loop                                                ▷ DDD iteration
    Thêm conflict clauses mới vào clause_store
    loop                                            ▷ RC2 loop
        solver ← NewSATSolver                       ▷ tạo mới mỗi SAT call
        for all clause c ∈ clause_store do solver.add_clause(c)
        end for
        result ← solver.solve_with_assumptions(rc2_state.soft_lits)
        Xử lý result, cập nhật rc2_state
        Nếu SAT: break
    end loop
end loop
```

7.4.3 Cấu hình C: Incremental SAT + Non-Incremental MaxSAT

- Một SAT solver duy nhất, giữ tất cả clauses.
- RC2 state được *reset* mỗi DDD iteration: xóa soft_constraints, đặt total_cost = 0.
- Tuy nhiên, Totalizer clauses đã thêm vào SAT solver *không thể xóa* (vì SAT solver là incremental) — cần dùng activation literals để "tắt" chúng.
- Ưu: RC2 sạch sẽ mỗi iteration, SAT solver giữ learned clauses.
- Nhược: phức tạp, cần activation literals cho Totalizer clauses.

Algorithm 6 Cấu hình C: Incremental SAT + Non-Incremental MaxSAT

```
solver ← NewSATSolver                                          ▷ tạo 1 lần duy nhất
loop                                                           ▷ DDD iteration
    solver.add_clause(conflict clauses mới)                    ▷ accumulative
    Tắt Totalizer clauses cũ (qua activation literals)
    rc2_state ← InitRC2                                        ▷ reset mỗi DDD iter
    loop                                                       ▷ RC2 loop — giải lại từ đầu
        result ← solver.solve_with_assumptions(rc2_state.soft_lits)
        Xử lý result, có thể thêm Totalizer clauses mới
        Nếu SAT: break
    end loop
end loop
```

7.4.4 Cấu hình D: Non-Incremental SAT + Non-Incremental MaxSAT

- Tạo SAT solver mới mỗi lần.
- RC2 state cũng reset mỗi DDD iteration.
- Ưu: đơn giản nhất, sạch sẽ nhất, preprocessing mạnh.
- Nhược: chi phí cao nhất, phải giải lại mọi thứ từ đầu.

7.5 Tổng hợp so sánh

Tiêu chí	**A (hiện tại)**	**B**	**C**	**D**
SAT incremental	Có	Không	Có	Không
MaxSAT incremental	Có	Có	Không	Không
Learned clauses	Giữ tất cả	Mất mỗi call	Giữ tất cả	Mất mỗi call
Totalizer state	Giữ	Giữ (rebuild)	Reset	Reset
Rebuild cost/iter	0	$O(\|H\|)$	0	$O(\|H\|)$
SAT calls/DDD iter	Ít	Ít	Nhiều	Nhiều
Preprocessing	1 lần	Mỗi call	1 lần	Mỗi call
Complexity	Thấp	Trung bình	Cao	Trung bình

Bảng 5: So sánh 4 cấu hình kết hợp SAT × MaxSAT incrementality

7.6 Đề xuất thực nghiệm

Thứ tự ưu tiên thử nghiệm:

1. **A** (baseline): cấu hình hiện tại, thiết lập reference.
2. **B** (Non-Incr SAT + Incr MaxSAT): thay đổi nhỏ nhất — chỉ cần tạo solver mới tại mỗi DDD iteration (hoặc mỗi SAT call), add lại clauses. RC2 state giữ nguyên. Cho phép đánh giá ảnh hưởng của learned clauses.
3. **C** (Incr SAT + Non-Incr MaxSAT): cần activation literals để tắt Totalizer cũ. Đánh giá xem RC2 state có bị "pollution" qua các DDD iteration không.
4. **D**: baseline cho "giải lại hoàn toàn mỗi iteration".

Metrics:

- Tổng thời gian giải, thời gian mỗi SAT call (trung bình/max).
- Số SAT calls (tổng và trong mỗi DDD iteration).
- Kích thước clause database tại mỗi iteration.
- Chất lượng lower bound qua các iteration.

8 So sánh hai biến thể encoding

Tiêu chí	One-Hot	Ladder (chính)
File	`maxsat_ddd.rs`	`maxsatddd_ladder.rs`
Biến/visit	$\lvert\Lambda^{ir}\rvert$ biến	$\lvert\Lambda^{ir}\rvert$ biến
Clause thêm 1 interval	$O(n)$ (AMO pairwise)	$O(1)$ (2 hard clause)
Exactly-one	Cần explicit	Implicit từ chuỗi monotone
Incompatibility	Binary clause (hard)	4-literal clause (hard)
Incremental	Tốn kém	**Hiệu quả**
SAT solver	MaxSatSolver trait	SatSolverWithCore trait

Bảng 6: So sánh hai biến thể encoding

9 Biến đổi sang MaxSAT Binary — Tối ưu bound lớn

9.1 Vấn đề với encoding hiện tại

Trong mô hình hiện tại, CostTree sử dụng **encoding đơn nguyên (unary)** cho chi phí:

- Mỗi mức chi phí c có một biến y_c và soft clause $\overline{y_c}$ với weight $= \Delta c$.
- Với `FiniteSteps123` (cost $\in \{0, 1, 2, 3\}$): chỉ 3 soft clauses/visit — **không vấn đề**.
- Với `InfiniteSteps180` (cost $= \lceil d/180 \rceil$): cost lên đến 30–60 $\rightarrow$ 30–60 soft clauses/visit.
- Với `Continuous` (cost $= d$ giây): cost lên đến hàng nghìn $\rightarrow$ hàng nghìn soft clauses/visit.

Hệ quả cho RC2. RC2 tìm UNSAT core và tăng lower bound bằng $w_{\min}$ của core. Với encoding unary (tất cả weight = 1), mỗi core chỉ tăng LB thêm 1. Ví dụ thực nghiệm (Instance A2, InfiniteSteps180):

$$\text{LB} = 1, 2, 3, \ldots, 37 \qquad \text{(37 UNSAT core iterations)}$$

Mỗi core iteration cần một SAT call $\Rightarrow$ tổng chi phí tỉ lệ thuận với OPT.

9.2 Ý tưởng: Binary Cost Encoding

Thay vì biểu diễn chi phí đơn nguyên, sử dụng **biểu diễn nhị phân**. Với chi phí tối đa C, chỉ cần $K = \lceil \log_2(C + 1) \rceil$ biến:

Cost type	Unary (hiện tại)	Binary (đề xuất)
FiniteSteps123 (max=3)	3 soft/visit	2 soft/visit
InfiniteSteps180 (max≈60)	60 soft/visit	6 soft/visit
InfiniteSteps60 (max≈180)	180 soft/visit	8 soft/visit
Continuous (max≈3600)	3600 soft/visit	12 soft/visit

9.3 Encoding nhị phân chi tiết

Cho mỗi visit (i, r):

Bước 1: Biến nhị phân. Tạo K biến $b_0^{ir}, b_1^{ir}, \ldots, b_{K-1}^{ir}$ với b_k biểu diễn bit thứ k của cost.

Bước 2: Soft clauses (S1'). Thay vì C soft clauses đơn nguyên, chỉ thêm K soft clauses:

Soft Clause

$(\overline{b_k^{ir}},\ w = 2^k)$ cho $k = 0, 1, \ldots, K-1$.

Ý nghĩa: "không muốn bit k của cost bằng 1" với weight 2^k.

Bước 3: Hard clauses liên kết delay → cost bits. Khi thêm time point t với cost c, cần encode: "nếu tàu delay đến t, thì cost $\geq c$". Với biểu diễn nhị phân, ta cần:

Cách 1: Giữ biến unary y_c nội bộ, liên kết sang binary. Giữ nguyên CostTree tạo các biến y_c unary nhưng **không tạo soft clause cho** y_c. Thay vào đó, liên kết:

Hard Clause

$y_c \Rightarrow b_k$ cho mỗi bit k mà c có bit thứ k bằng 1.

Với cách này:

- Hard clauses H4 $(\overline{d_t} \vee y_c)$ và H5 $(y_{c'} \Rightarrow y_c)$ giữ nguyên.
- Soft clauses S1 thay bằng S1': $\overline{b_k}$ weight 2^k.
- Thêm hard clause: $\overline{y_c} \vee b_k$ cho mỗi k mà bit k của c bằng 1.

Vấn đề: encoding này *không chính xác* vì b_k là bit, không phải ngưỡng. Nếu cost $= 5$ (binary 101), việc $b_0 = 1$ chỉ có nghĩa "cost có bit 0", không phải "cost ≥ 1". Cần phân tích kỹ hơn.

Cách 2: Binary encoding trực tiếp cho tổng cost. Encode tổng cost toàn cục $\sum_{i,r} c^{ir}$ bằng binary, thay vì encode per-visit.

1. Mỗi visit vẫn dùng unary y_c theo CostTree hiện tại.
2. Thay vì soft clause $\overline{y_c}$ weight Δc, sử dụng **adder circuit** (Pseudo-Boolean to CNF):
   - Tổng hợp tất cả cost variables thành biểu diễn binary: $\sum_{\text{visit}} \text{cost}^{\text{visit}} = (B_{K-1}, \ldots, B_0)_2$.
   - Soft clauses trên các bit tổng: $\overline{B_k}$ weight 2^k.

Đây thực chất là encoding **Pseudo-Boolean (PB) constraint** trên hàm mục tiêu.

Cách 3: Weight stratification cải tiến trong RC2. Thay vì thay đổi encoding, thay đổi **cách RC2 xử lý weight**:

- **Weight stratification nhị phân:** Thay vì xử lý weight w bằng cách trừ $w_{\min}$ (có thể $= 1$), nhóm soft clauses theo lũy thừa 2 và xử lý từ nhóm cao nhất.

- Cụ thể: mỗi soft clause $(\overline{y_c}, w)$ được thay bằng $\lfloor \log_2 w \rfloor + 1$ copies:

$$(\overline{y_c^{(k)}}, 2^k) \quad \text{cho mỗi bit } k \text{ mà } w \text{ có bit } k = 1$$

  với hard clause $y_c^{(k)} \Leftrightarrow y_c$ (dùng cùng literal, chỉ tách weight).

- Với encoding hiện tại (weight $\Delta c = 1$ cho mỗi mức cost), cách này **không giúp gì** vì weight đã là 1.

9.4 Cách tiếp cận khả thi nhất: Binary CostTree

Thay đổi CostTree để sử dụng ngưỡng nhị phân thay vì đơn nguyên:

1. Thay vì tạo biến y_c cho mỗi mức cost $c = 1, 2, 3, \ldots$, tạo biến cho ngưỡng lũy thừa 2:

$$y_1^{ir},\ y_2^{ir},\ y_4^{ir},\ y_8^{ir},\ \ldots,\ y_{2^{K-1}}^{ir}$$

   với $y_{2^k} = 1$ iff cost $\geq 2^k$.

2. Soft clauses:

   Soft Clause

   $(\overline{y_{2^k}^{ir}},\ w = 2^k) \quad$ cho $k = 0, 1, \ldots, K-1$.

3. Hard clauses: khi thêm time point t với cost c:

   Hard Clause

   $\overline{d_t^{ir}} \vee y_{2^k}^{ir} \quad$ cho mỗi k sao cho $2^k \leq c$.

   Ý nghĩa: delay đến t implies cost $\geq 2^k$ cho tất cả $2^k \leq c$.

4. Chain ordering:

   Hard Clause

   $\overline{y_{2^{k+1}}^{ir}} \vee y_{2^k}^{ir} \quad$ (cost $\geq 2^{k+1}$ implies cost $\geq 2^k$).

Ví dụ. Cost $c = 5$ (binary 101):

- $c \geq 1$ (2^0): $\overline{d_t} \vee y_1$ ✓
- $c \geq 2$ (2^1): $\overline{d_t} \vee y_2$ ✓
- $c \geq 4$ (2^2): $\overline{d_t} \vee y_4$ ✓

Soft clauses $\overline{y_1}$ (weight 1), $\overline{y_2}$ (weight 2), $\overline{y_4}$ (weight 4). Nếu cả 3 bị vi phạm: cost tối thiểu $= 1 + 2 + 4 = 7 \geq 5$. Đây là *over-approximation* nhưng vẫn đúng vì cost thực sự ≥ 5, và MaxSAT sẽ tối thiểu hóa tổng weight bị vi phạm.

Tính đúng đắn. Encoding binary là một **relaxation**: giá trị cost được "làm tròn lên" thành tổng các lũy thừa 2. Cụ thể:

- Nếu cost thực $= c$, thì encoding binary enforce rằng tổng weight vi phạm $\geq c$ (vì $c \leq \sum_{k:2^k \leq c} 2^k = 2^{\lfloor \log_2 c \rfloor + 1} - 1$).
- Solver sẽ tìm assignment tối ưu trên binary encoding, LB vẫn hợp lệ.
- Tuy nhiên, cost tối ưu trên binary encoding *có thể lớn hơn* cost tối ưu thực — đây là over-estimation.

Visit cost	**Unary exact**	**Binary approx**	**Sai số**
$c = 1$	1	1	0
$c = 2$	2	$1 + 2 = 3$	+1
$c = 3$	3	$1 + 2 = 3$	0
$c = 5$	5	$1 + 2 + 4 = 7$	+2
$c = 10$	10	$1 + 2 + 4 + 8 = 15$	+5

Bảng 7: So sánh cost chính xác vs binary approximation

Giải pháp cho over-estimation. Để tránh sai số, có thể dùng **Mixed Binary-Unary** encoding:

1. Với cost thấp ($c \leq T$): dùng unary (chính xác).
2. Với cost cao ($c > T$): dùng binary (xấp xỉ, nhưng ít soft clauses).
3. Ngưỡng T chọn sao cho unary part có $\leq T$ soft clauses.

Hoặc sử dụng **Generalized Totalizer Encoding (GTE)** cho PB constraints, encode chính xác hàm mục tiêu bằng biểu diễn nhị phân mà không mất chính xác.

9.5 Tác động lên RC2

	Unary (hiện tại)	**Binary**	**Mixed**
Soft clauses/visit	C	$\log_2 C$	$T + \log_2(C/T)$
Min weight	1	1	1
Max weight	$\Delta c_{\max}$	2^{K-1}	2^{K-1}
RC2 cores cho OPT=N	$\geq N$	$O(\log N)$	$O(T + \log N)$
Chính xác	Có	Xấp xỉ	Có (nếu GTE)

Bảng 8: So sánh tác động lên RC2

Lợi ích chính. Với InfiniteSteps180, Instance A2 (cost tối ưu = 37):

- Unary: 37 RC2 core iterations (mỗi lần +1).
- Binary: ≈ 6 RC2 core iterations (weight $32, 4, 1 \Rightarrow$ LB jumps lớn hơn).

Weight stratification trong RC2 tự động ưu tiên soft clauses có weight cao $\Rightarrow$ xử lý bit cao trước, tăng LB nhanh hơn.

10 Thay thế MaxSAT bằng Incremental SAT + SCPB

10.1 Tại sao không dùng Totalizer?

Hàm mục tiêu là PB constraint, không phải Cardinality. Trong mô hình hiện tại, CostTree tạo soft clauses $(\overline{y_c^{ir}}, w)$ trong đó weight $w = \Delta c$ là hiệu cost giữa các mức liên tiếp. Cụ thể:

- **Chế độ linear** (CostTree, khi dist_prev < 10): mỗi mức cost tạo 1 biến, $\Delta c = 1 \rightarrow$ weight $= 1$ (cardinality).
- **Chế độ tree** (CostTree, khi dist_prev ≥ 10): jump qua nhiều mức, $\Delta c > 1 \rightarrow$ weight > 1 (**PB constraint**).

Bài báo gốc cũng xác nhận: hàm mục tiêu $\min \sum_{i \in N} w_i x_i$ với w_i thay đổi tùy cost type. Bài báo nhận xét: *"When weights vary a lot... representing this trade-off exactly as SAT constraints is computationally expensive."*

> **Kết luận:** Hàm mục tiêu là **Pseudo-Boolean constraint** $\sum w_i X_i \leq k$ với $w_i \geq 1$. Totalizer chỉ hỗ trợ cardinality ($w_i = 1$). $\Rightarrow$ Phải dùng **SCPB (Sequential Counter for PB)**.

10.2 SCPB Encoding

Cho n biến cost $X_1, \ldots, X_n$ với trọng số $w_1, \ldots, w_n$, SCPB sử dụng **biến đếm nhị phân** $R_1, \ldots, R_{n-1}$:

- R_i gồm $\text{pos}_i = \min(k, \sum_{j=1}^{i} w_j)$ bits: $\{R_{i,1}, R_{i,2}, \ldots, R_{i,\text{pos}_i}\}$.
- **Ngữ nghĩa:** Khi $\sum_{j=1}^{i} w_j X_j = p$, thì đúng p bits đầu tiên của R_i có giá trị 1.

Các công thức encoding.

$$\bigwedge_{i=1}^{n-1} \bigwedge_{j=1}^{w_i} X_i \longrightarrow R_{i,j} \tag{1}$$

$$\bigwedge_{i=2}^{n-1} \bigwedge_{j=1}^{\text{pos}_{i-1}} R_{i-1,j} \longrightarrow R_{i,j} \tag{2}$$

$$\bigwedge_{i=2}^{n-1} \bigwedge_{j=1}^{\text{pos}_{i-1}} X_i \wedge R_{i-1,j} \longrightarrow R_{i,j+w_i} \tag{3}$$

$$\bigwedge_{i=2}^{n-1} \bigwedge_{j=1}^{\text{pos}_{i-1}} \neg X_i \wedge \neg R_{i-1,j} \longrightarrow \neg R_{i,j} \tag{4}$$

(khi $\text{pos}_{i-1} < k$)

$$\bigwedge_{i=1}^{n-1} \bigwedge_{j=1+\text{pos}_{i-1}}^{\text{pos}_i} \neg X_i \longrightarrow \neg R_{i,j} \tag{5}$$

$$\bigwedge_{i=2}^{n-1} \bigwedge_{j=1}^{\text{pos}_{i-1}} \neg R_{i-1,j} \longrightarrow \neg R_{i,j+w_i} \tag{6}$$

AMK encoding ($\sum w_i X_i \leq k$). Thêm công thức (8):

$$\bigwedge_{i=2}^{n} X_i \longrightarrow \neg R_{i-1,k+1-w_i} \tag{8}$$

10.3 Điều khiển bound qua Assumptions

Key insight: Biến $R_{n-1,k}$ encode trực tiếp "$\sum_{j=1}^{n-1} w_j X_j \geq k$":

Hard Clause

- Enforce $\sum w_i X_i \leq k$: thêm **assumption** $\overline{R_{n-1,k+1}}$ khi gọi SAT solver.
- Enforce $\sum w_i X_i \geq k$: thêm assumption $R_{n-1,k}$.
- Enforce $\sum w_i X_i \in [u, v]$: assumptions $R_{n-1,u}$ và $\overline{R_{n-1,v+1}}$.

Thay đổi bound $k \to k'$ chỉ cần đổi assumption, không thêm/xóa clause nào.
So sánh với Totalizer (chỉ hỗ trợ cardinality):

	Totalizer	**SCPB**
Hỗ trợ weights	Không ($w_i = 1$)	**Có** ($w_i \geq 1$)
Variables	$O(n \log n)$	$O(nk)$
Tăng bound	Thêm clauses	**Chỉ đổi assumption**
Giảm bound	Không thể	**Chỉ đổi assumption**

10.4 Áp dụng cho hàm mục tiêu

Bước 1: Thu thập cost literals. Gọi $\mathcal{Y} = \{(y_c^{ir}, w_c^{ir})\}$ là tập tất cả cost literals với trọng số tương ứng:

- y_c^{ir} = biến cost cho visit (i, r) tại mức c (từ CostTree).
- $w_c^{ir} = \Delta c$ = hiệu cost giữa mức c và mức cost thấp hơn kế tiếp.

Hàm mục tiêu: $\min \sum_{(y,w)\in\mathcal{Y}} w \cdot y$.

Bước 2: Tạo SCPB.

$$\text{SCPB}(\{y_c^{ir}\}, \{w_c^{ir}\}, k_{\max})$$

Tạo biến đếm $R_1, \ldots, R_{|\mathcal{Y}|-1}$, mỗi R_i có tối đa $k_{\max}$ bits. Thêm các clauses (1)–(6) vào SAT solver.

Algorithm 7 SAT + SCPB: Binary Search

```
𝒴 ← tập cost literals {(y_c^ir, w_c^ir)}
Tạo SCPB từ 𝒴 với k_max = UB (từ heuristic)
lo ← 0, hi ← UB
while lo < hi do
    k ← (lo + hi)/2
    result ← solver.solve_with_assumptions(R̄_{n-1,k+1})
    if SAT then
        hi ← cost thực tế từ model
    else
        lo ← k + 1
    end if
end while
return lo
```

Bước 3: Tìm optimal bằng binary search. **Số SAT calls** $= O(\log(\text{UB} - \text{LB}))$. So với RC2 cần $O(\text{OPT})$ calls.

SAT-UNSAT Search (từ trên xuống). Nếu có UB tốt từ heuristic:

1. Bắt đầu với $k = \text{UB}$.
2. Gọi SAT với assumption $\overline{R_{n-1,k}}$ (tức $\sum \leq k - 1$).
3. Nếu SAT: $k \leftarrow$ cost thực từ model, tiếp tục.
4. Nếu UNSAT: k là optimal.

10.5 Tích hợp với DDD

Algorithm 8 DDD + SAT + SCPB (thay thế RC2)

```
solver ← SAT solver mới
𝒴 ← ∅                                          ▷ tập cost literals, mở rộng dần
scpb ← None
loop                                            ▷ DDD iteration
    if SAT ở iteration trước then
        Phát hiện conflicts → thêm hard clauses H1–H3
        if không conflict then return optimal
        end if
    end if
    Thêm time points mới:
    for all time point mới t cho visit (i, r) với cost c > 0 do
        Thêm hard H4: d̄_t ∨ y_c
        Thêm hard H5: y_c' ⇒ y_c
        if y_c mới then
            𝒴 ← 𝒴 ∪ {(y_c, w_c)}
            scpb.extend(y_c, w_c)                ▷ mở rộng SCPB
        end if
    end for
    Tìm k tối ưu: binary search trên k với assumption R̄_{n-1,k+1}
end loop
```

Mở rộng SCPB incrementally. Khi DDD thêm time point mới $\rightarrow$ cost literal mới (y_c, w_c) $\rightarrow$ cần thêm input vào SCPB:

1. **Thêm input mới X_{n+1} với weight w_{n+1}:**
   - Tạo biến đếm mới R_n với $\text{pos}_n = \min(k, \text{pos}_{n-1} + w_{n+1})$ bits.
   - Thêm clauses (1)–(3) cho $i = n$:
     - (1): $X_{n+1} \Rightarrow R_{n,j}$ cho $j \in [1, w_{n+1}]$.
     - (2): $R_{n-1,j} \Rightarrow R_{n,j}$ cho $j \in [1, \text{pos}_{n-1}]$.
     - (3): $X_{n+1} \wedge R_{n-1,j} \Rightarrow R_{n,j+w_{n+1}}$.
   - Thêm clause (8) cho AMK: $X_{n+1} \Rightarrow \overline{R_{n-1,k+1-w_{n+1}}}$.
2. **Cập nhật assumption:** $\overline{R_{n,k+1}}$ thay vì $\overline{R_{n-1,k+1}}$ (dùng biến đếm mới nhất).
3. **Số clauses thêm mới**: $O(\text{pos}_{n-1})$ cho mỗi input mới, cấu trúc tuần tự thuận lợi cho incremental.

10.6 Hybrid SCPB (SCPB^h)

Giá trị k ảnh hưởng số bits của R_i. Khi k lớn (gần totalW $= \sum w_i$), normalize bằng cách đảo:

$$\text{SCPB}^h_{\leq k} \equiv \begin{cases} \text{SCPB}_{\leq k} & \text{nếu } k \text{ nhỏ (so với totalW)}, \\ \text{SCPB}_{\geq (n-k)} \text{ (trên } \neg X) & \text{nếu } k \text{ lớn (gần totalW)}. \end{cases}$$

Ví dụ: $3X_1 + 5X_2 + 7X_3 + 2X_4 + 6X_5 + 4X_6 \geq 22$ (totalW $= 27$) $\equiv 3\overline{X_1} + 5\overline{X_2} + 7\overline{X_3} + 2\overline{X_4} + 6\overline{X_5} + 4\overline{X_6} \leq 5$. Mỗi R_i cần 5 bits thay vì 22 bits.

10.7 So sánh RC2 vs SAT + SCPB

Tiêu chí	**RC2 (hiện tại)**	**SAT + SCPB**
Loại solver	MaxSAT (core-guided)	Pure SAT
Hỗ trợ PB	Gián tiếp (weight trong soft)	**Trực tiếp** (encoding PB)
Tìm optimal	UNSAT cores, LB $+= w_{\min}$	Binary search trên k
SAT calls (unit w)	$=$ OPT	$O(\log \text{OPT})$ (binary search)
SAT calls (có UB)	$=$ OPT $-$ LB$_0$	$O(\log(\text{UB} - \text{LB}))$
Thay đổi bound	Thêm clauses	**Đổi assumption**
Encoding overhead	Totalizer/core (nhỏ)	SCPB: $O(nk)$ biến/clauses
Learned clauses	Giữ qua iterations	Giữ qua iterations
DDD refinement	Thêm soft clause	Thêm R_n, $O(\text{pos})$ clauses
Khó khăn chính	Nhiều cores nếu OPT lớn Weights khác nhau $\rightarrow$ chậm	SCPB lớn ($n \times k$ biến) Cần biết $k_{\max}$ trước

Bảng 9: So sánh RC2 vs SAT + SCPB

Khi nào SAT + SCPB tốt hơn:

- **OPT lớn** (InfiniteSteps, Continuous): binary search $= O(\log \text{OPT})$ vs RC2 $= O(\text{OPT})$.
- **Có UB tốt** từ heuristic: search space nhỏ $\rightarrow$ rất ít SAT calls.
- **Weights khác nhau**: SCPB xử lý trực tiếp, RC2 gặp khó khăn.
- **DDD refinement**: thêm input $O(\text{pos})$ clauses, giữ learned clauses.

Khi nào RC2 tốt hơn:

- **OPT nhỏ** (Stepwise): RC2 chỉ cần vài cores, SCPB overhead lớn hơn.
- Không biết UB trước (SCPB cần $k_{\max}$ để tạo encoding).
- $n \times k$ quá lớn (quá nhiều biến counter R).

10.8 Generalized Totalizer Encoding (GTE)

SCPB (Mục trước) sử dụng sequential counter — encoding tuần tự với $O(nk)$ biến phụ. Một phương pháp thay thế hiệu quả hơn là **Generalized Totalizer Encoding** (GTE) [1], mở rộng Totalizer cho PB constraints.

Ý tưởng GTE. Totalizer chuẩn xây dựng cây nhị phân cho n inputs, mỗi node tổng hợp cardinality ($w_i = 1$). GTE tổng quát hóa:

- Mỗi lá có trọng số $w_i \geq 1$.
- Mỗi node nội bộ tổng hợp **weighted sum** của hai con.
- Output variables tại root encode các mức "$\sum w_i X_i \geq v$" cho từng giá trị v khả thi.

So sánh encoding.

	SCPB	**GTE**	**DPW**
Cấu trúc	Tuần tự (chain)	Cây nhị phân	Watchdog
Biến phụ	$O(nk)$	$O(n \log n \cdot W)$	$O(n \log^2 n)$
Clauses	$O(nk)$	**Ít hơn** (lazy)	$O(n \log^2 n)$
Incremental add	Có	**Có**	Có
Incremental bound	Assumption	**Assumption**	Assumption
Lazy encoding	Không	**Có**	Có

(W = max weight; DPW = Dynamic Poly Watchdog, một phương pháp thay thế khác.)

API (RustSAT). Thư viện `rustsat` (Rust) cung cấp `GeneralizedTotalizer` với API incremental:

1. **Tạo:** `let mut gte = GeneralizedTotalizer::from(lits_weights);`
   `lits_weights`: `HashMap<Lit, usize>` — mapping literal → weight.
2. **Thêm input incremental:** `gte.extend([(new_lit, weight)]);`
   Tương ứng DDD thêm time point mới → cost literal mới.
3. **Encode bound:** `gte.encode_ub(0..=k, &mut cnf, &mut vars);`
   Lazy: chỉ sinh clauses cho range $[0, k]$. Nếu gọi lại với $k' > k$, chỉ sinh thêm phần mới.
4. **Lấy assumptions:** `let assumps = gte.enforce_ub(k);`
   Trả về `Vec<Lit>` — truyền trực tiếp vào `solver.solve_with_assumptions(assumps)`.
5. **Thay đổi bound:** Gọi `encode_ub` + `enforce_ub` với k' mới. **Không cần reset solver.**

Algorithm 9 DDD + SAT + GTE (thay thế RC2)

```
solver ← SAT solver mới
gte ← GeneralizedTotalizer rỗng
loop                                                    ▷ DDD iteration
    if SAT ở iteration trước then
        Phát hiện conflicts → thêm hard clauses
        if không conflict then return optimal
        end if
    end if
    for all time point mới (i, r, c) với cost c > 0 do
        gte.extend([(y_c, w_c)])                        ▷ thêm cost literal
    end for
    Binary search trên k:
    gte.encode_ub(0..=k, cnf, vars)                     ▷ lazy, chỉ sinh thêm
    assumps ← gte.enforce_ub(k)
    result ← solver.solve(assumps)
end loop
```

Tích hợp với DDD.

Ưu điểm GTE so với SCPB:

- **Lazy encoding**: chỉ sinh clauses khi cần, giảm overhead cho bound nhỏ.
- **Cấu trúc cây**: propagation tốt hơn chain (SCPB).
- **Production-ready**: thư viện `rustsat` đã benchmark, ít clauses hơn PbLib.
- **Proof logging**: hỗ trợ VeriPB certification.
- Không cần biết $k_{\max}$ trước (lazy).

Lưu ý tương thích. Project hiện tại sử dụng `satcoder` crate (literal type riêng). `rustsat` sử dụng `rustsat::types::Lit`. Cần adapter hoặc chuyển sang `rustsat` solver interface.

11 Tổng kết

Mô hình MaxSAT DDD cho Train Rescheduling có cấu trúc rõ ràng:

- **Hard constraints** đảm bảo tính khả thi vật lý: H1 (delay chain), H2 (travel time), H3 (resource conflict), H4–H5 (cost implication/ordering), H6 (encoding internal).
- **Soft constraints** mã hóa mục tiêu: S1 (cost minimization), S2 (Totalizer bound từ RC2).
- **RC2** tìm UNSAT cores, nới lỏng bằng Totalizer, tăng LB dần.

Hàm mục tiêu là **PB constraint** ($\sum w_i X_i \leq k$, $w_i \geq 1$) do CostTree tạo soft clauses với weight $= \Delta c > 1$ ở chế độ tree.

Các hướng cải tiến:

1. **Hai tầng incremental** (Mục 7): 4 cấu hình SAT $\times$ MaxSAT.
2. **Binary MaxSAT** (Mục 9): giảm soft clauses $O(C) \to O(\log C)$.

3. **SAT + PB encoding** (Mục 10): thay MaxSAT bằng SAT + PB encoding (SCPB hoặc GTE), binary search $O(\log \mathrm{OPT})$, điều khiển bound qua assumptions.
   - **SCPB**: sequential counter, $O(nk)$ biến, tự implement.
   - **GTE** (Mục 10.8): generalized totalizer, lazy encoding, production-ready (`rustsat`).

Tài liệu

[1] S. Joshi, R. Martins, V. Manquinho. Generalized Totalizer Encoding for Pseudo-Boolean Constraints. *Proceedings of CP 2015*, pp. 200–209, 2015.

[2] O. Bailleux, Y. Boufkhad, O. Roussel. New Encodings of Pseudo-Boolean Constraints into CNF. *Proceedings of SAT 2009*, pp. 181–194, 2009.

[3] C. Jabs, A. Berg, M. Järvisalo, J. Ihalainen. RustSAT: A Modular Library for SAT Solving and Constraint Encoding. *Proceedings of SAT 2024*, 2024.

[4] B. Luteberget, G. Kaasen. A MaxSAT Approach for Solving a New Dynamic Discretization Discovery Model for Train Rescheduling Problems. 2023.